AI Usage Log

Student: Hind Houfaidi
SID: 40125306

Task ID: SCRUM-173: Improve Job Posting API Validation and Error Handling

Purpose of AI Use: To handle malformed MongoDB IDs in Express job routes.

Prompt: In Express and Mongoose, how can I use mongoose.isValidObjectId() to return HTTP 400 for invalid IDs in GET, PUT, and DELETE /api/jobs/:id routes? How can I test these cases with Jest and Supertest?

Response: AI suggested validating the route parameter before querying MongoDB and adding an integration test for each route.

AI-Suggested Content: ObjectId validation middleware and test cases for malformed job IDs.

Validation: Reviewed the route and test changes. Local backend tests passed, and PR #34 was merged into main.

Decision: Partially accepted and modified before use.

Reflection: AI helped clarify the difference between malformed IDs and valid IDs that do not exist.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-174: Recruiter Application Inbox

Purpose of AI Use: To understand the backend endpoint and frontend integration for recruiter applications.

Prompt: In Express and Mongoose, how can I build GET /api/applications/jobs/:jobId to return applications with applicant details, check that the logged-in recruiter owns the posting, and avoid returning the applicant password?

Response: AI suggested an authenticated route, a job ownership check, Mongoose population of applicant information, and a frontend API call.

AI-Suggested Content: Suggested API logic, ownership checks, and frontend integration steps.

Validation: Reviewed the implementation and integration tests. PR #31 was merged into main; later backend integration tests passed.

Decision: Partially accepted and modified before use.

Reflection: AI helped connect the recruiter dashboard to application records while considering access control.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-175: First-Come, First-Served Application Handling

Purpose of AI Use: To test chronological application ordering in the recruiter inbox.

Prompt: Using Jest, Supertest, and Mongoose, how can I create three applications with different createdAt timestamps, insert them in reverse order, and assert that GET /api/applications/jobs/:jobId returns the oldest application first?

Response: AI suggested fixed timestamps and comparing the returned application IDs with the expected chronological order.

AI-Suggested Content: Integration test approach using controlled createdAt values and an ordering assertion.

Validation: Reviewed and ran the integration tests. PR #33 was merged into main. The later backend run passed 40/40 integration tests.

Decision: Accepted with review.

Reflection: AI helped me test ordering by timestamps rather than relying on insertion order.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-146: Recruiter Dashboard Frontend Test Bug (PR #35)

Purpose of AI Use: To fix a failing React Testing Library assertion in GitHub Actions.

Prompt: In React Testing Library, why does screen.getByText("Posted today") fail when the component renders "Posted today · Updated today"? Can I use screen.getByText(/Posted today/) without changing the component?

Response: AI explained that exact matching was too restrictive and suggested a regular-expression assertion.

AI-Suggested Content: Replace the exact text assertion with screen.getByText(/Posted today/).

Validation: Ran frontend unit tests locally: 35/35 passed across 8 suites. The one-line test fix was merged in PR #35.

Decision: Accepted.

Reflection: AI helped fix a brittle test without changing application behavior.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-218: Test MongoDB Integration and Data Persistence

Purpose of AI Use: To execute database integration tests and identify which database operations they cover.

Prompt: In a Jest and Mongoose project using MongoMemoryServer, which command runs the integration test suite, and how can I verify that create, update, and delete requests actually change JobPosting documents in the test database?

Response: AI suggested running the existing integration suite and checking database records with Mongoose queries, while keeping Atlas persistence verification separate.

AI-Suggested Content: Integration test command and database assertions such as JobPosting.findById().

Validation: Ran backend integration tests: 40/40 passed across 3 suites using MongoMemoryServer. MongoDB Atlas persistence and screenshots are still pending.

Decision: Partially accepted.

Reflection: AI helped distinguish temporary test database coverage from verification in MongoDB Atlas.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-217: Test Recruiter Application Inbox Functionality

Purpose of AI Use: To check the application endpoint tests and frontend test execution.

Prompt: With Jest and Supertest, how can I test that GET /api/applications/jobs/:jobId returns HTTP 200 for the owning recruiter, 403 for another recruiter or a job seeker, 401 without authentication, and applications sorted by createdAt?

Response: AI identified endpoint assertions for access control, application ordering, and safe applicant fields, along with test commands.

AI-Suggested Content: Expected HTTP statuses, ordering assertions, and backend/frontend test commands.

Validation: Backend integration tests passed (40/40), and frontend unit tests passed (35/35). Manual browser testing of the inbox is pending.

Decision: Partially accepted.

Reflection: AI helped verify expected API behavior and separate automated checks from browser testing.

Responsible Person: Hind Houfaidi

Task ID: SCRUM-216: Test the Whole Implementation

Purpose of AI Use: To execute automated tests for the integrated recruiter functionality.

Prompt: What npm commands should I run to execute the Jest backend integration tests and React frontend unit tests separately, and how can I tell whether jsdom navigation and React act(...) console warnings mean the test suite failed?

Response: AI provided the backend and frontend Jest commands and explained that the observed console warnings were nonfatal when the test suites passed.

AI-Suggested Content: Commands for both test suites and guidance for interpreting Jest output.

Validation: Verified terminal results: 40/40 backend integration tests and 35/35 frontend unit tests passed. End-to-end browser testing remains pending.

Decision: Partially accepted.

Reflection: AI helped interpret the test output without treating warnings as failed tests.

Responsible Person: Hind Houfaidi